



CL1002

***Programming
Fundamentals Lab***

Lab 10

Nested Structures,
Composition and
Structure array and
Filing in C

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES

Fall 2024

AIMS AND OBJECTIVES**Aim:**

To understand and implement nested structures, structure composition, arrays of structures, and file handling to manage and organize complex data in C programming.

Objectives:**Introduction**

Section 1: Structures

The structure in C is a user-defined data type that can be used to group items of possibly different types into a single type. The **struct** keyword is used to define the structure in the C programming language. The items in the structure are called its **member** and they can be of any valid data type. Additionally, the values of a structure are stored in contiguous memory locations like arrays. The difference between an array and a structure is that an array is a homogenous collection of similar types, whereas a structure can have elements of different types stored adjacently and identified by a name.

1.1 Declaration of Struct

```
struct Chocolate{  
    char Name[20];  
    float Weight;  
    int Calories;  
    float Price;  
    char ExpiryDate[10];  
};
```

1.2 Declaration & Initialization of Struct type Variables

You can declare the variables before the semi-colon(;) or using a proper declaration syntax like other

variable's in main();

```
struct Chocolate{
char Name[20];
float Weight;
int Calories;
float Price;
char ExpiryDate[10];
}var1, var2,var3;
```

```
Int main()
{
Struct Chocolate Kitkat, Mars, Jubilee, mychocolate[3];
}
```

// OR

```
struct Chocolate myChocolate;
gets(myChocolate.Name);
myChocolate.Weight= 20;
myChocolate.Calories= 500;
myChocolate.Price= 100;
strcpy(myChocolate.ExpiryDate,"01-Feb-2021");
// OR
struct Chocolate Jubilee = {"Jubilee",20.50,500,100,"01-Feb-2021"};
}
```

1.3 Declaration & Initialization of Struct type Array

```
Int main()
{
// Array of Struct
struct Chocolate myFavChocolates[3]; // It is an array of struct
int i = 0;
while(i<3)
{
gets(myChocolate[i].Name);
scanf("%f",&myChocolate.Weight);
scanf("%d",&myChocolate.Calories);
scanf("%f",&myChocolate.Price);
gets(myChocolate[i].ExpiryDate);
++i;
}
// TO print this array of Struct
i = 0;
while(i<3)
{
```

```
puts(myChocolate[i].Name);  
printf("%f",myChocolate.Weight);  
printf("%d",myChocolate.Calories);  
printf("%f",myChocolate.Price);  
puts(myChocolate[i].ExpiryDate);  
++i;  
}  
}
```

2.0 Nested Structures

Nested structure in C is nothing but structure within structure. One structure can be declared inside other structure as we declare structure members inside a structure. The structure variables can be a normal structure variable ,array or a pointer variable to access the data. You can learn below concepts in this section.

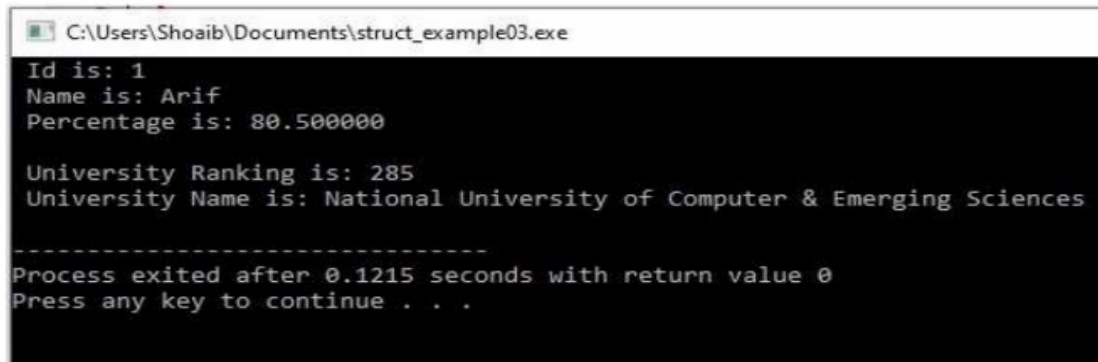
```
#include <stdio.h>
#include <string.h>

struct UniversityDetails
{
    int UniversityRanking;
    char UniversityName[90];
};

struct student_detail
{
    int id;
    char name[20];
    float percentage;
    // structure within structure
    struct UniversityDetails data;
};

int main()
{
    struct student_detail std_data = {1, "Arif", 80.5, 285,
                                      "National University of Computer & Emerging Sciences"};
    printf(" Id is: %d \n", std_data.id);
    printf(" Name is: %s \n", std_data.name);
    printf(" Percentage is: %f \n\n", std_data.percentage);

    printf(" University Ranking is: %d \n",
           std_data.data.UniversityRanking);
    printf(" University Name is: %s \n",
           std_data.data.UniversityName);
    return 0;
}
```

OUTPUT:

```
C:\Users\Shoaib\Documents\struct_example03.exe
Id is: 1
Name is: Arif
Percentage is: 80.500000

University Ranking is: 285
University Name is: National University of Computer & Emerging Sciences

-----
Process exited after 0.1215 seconds with return value 0
Press any key to continue . . .
```

Another Examples of nested structures

Sample Code:

```

#include <stdio.h>
#include <string.h>

struct Type{
    char TypeName[20];    // Mini, Sedan, Sports, Luxury, SUV
};

struct Car{
    char CarName[20];
    char make[15];
    char model[15];
    char color[10];
    int seats;
    int engine;           // 1800 cc
    int price;

    struct Type CarType;
};

int main()
{
    struct Car myCar;

    puts("----- Example: Nested Structure -----");

    puts("Enter the Name of your Car: ");
    gets(myCar.CarName);
    puts("Enter the type of your Car (Mini, Sedan, Sports, Luxury, SUV): ");
    gets(myCar.CarType.TypeName);

    puts("Enter the Color of your Car: ");
    gets(myCar.color);
    puts("Enter the make of your Car: ");
    gets(myCar.make);
    puts("Enter the model of your Car: ");
    gets(myCar.model);
    printf("\nEnter the seats of your Car: ");
    scanf("%d",&myCar.seats);
    printf("\nEnter the engine capacity (cc) of your Car: ");
    scanf("%d",&myCar.engine);
    printf("\nEnter the price of your Car: ");
    scanf("%d",&myCar.price);

    puts("\n\n----- Print -----");

    printf("\nCarName: %s",myCar.CarName);
    printf("\nCarType: %s",myCar.CarType.TypeName);
    printf("\nColor: %s",myCar.color);
    printf("\nMake: %s",myCar.make);
    printf("\nModel: %s",myCar.model);
    printf("\nSeats: %d",myCar.seats);
    printf("\nEngine (cc): %d",myCar.engine);
    printf("\nPrice: %d", myCar.price);

    return 0;
}

```

OUTPUT:

```
C:\Users\Shoaib\Documents\struct_example02.exe
----- Example: Nested Structure -----
Enter the Name of your Car:
Picanto 2.0
Enter the type of your Car {Mini, Sedan, Sports, Luxury, SUV}:
Mini
Enter the Color of your Car:
White
Enter the make of your Car:
KIA
Enter the model of your Car:
Picanto

Enter the seats of your Car: 4

Enter the engine capacity (cc) of your Car: 1300

Enter the price of your Car: 120000

----- Print -----

CarName: Picanto 2.0
CarType: Mini
Color: White
Make: KIA
Model: Picanto
Seats: 4
Engine (cc): 1300
Price: 120000
-----
Process exited after 54.59 seconds with return value 0
Press any key to continue . . .
```

C File Handling

file is a container in computer storage devices used for storing data.

Why files are needed?

- When a program is terminated, the entire data is lost. Storing in a file will preserve your data even if the program terminates.
- If you have to enter a large number of data, it will take a lot of time to enter them all.
- However, if you have a file containing all the data, you can easily access the contents of the file using a few commands in C.
- You can easily move your data from one computer to another without any changes.

Types of Files

When dealing with files, there are two types of files you should know about:

1. Text files
2. Binary files

1. Text files

Text files are the normal .txt files. You can easily create text files using any simple text editors such as Notepad.

When you open those files, you'll see all the contents within the file as plain text. You can easily edit or delete the contents.

They take minimum effort to maintain, are easily readable, and provide the least security and takes bigger storage space.

2. Binary files

Binary files are mostly the .bin files in your computer.

Instead of storing data in plain text, they store it in the binary form (0's and 1's).

They can hold a higher amount of data, are not readable easily, and provides better security than text files.

File Operations

In C, you can perform four major operations on files, either text or binary:

1. Creating a new file
2. Opening an existing file
3. Closing a file
4. Reading from and writing information to a file

Working with files

When working with files, you need to declare a pointer of type file. This declaration is needed for communication between the file and the program.

```
FILE *fptr;
```

Opening a file - for creation and edit

Opening a file is performed using the `fopen()` function defined in the `stdio.h` header file.

The syntax for opening a file in standard I/O is:

```
ptr = fopen("filename", "mode");
```

For example,

```
fopen("E:\\cprogram\\newprogram.txt", "w");
```

```
fopen("E:\\cprogram\\oldprogram.bin", "rb");
```

- Let's suppose the file `newprogram.txt` doesn't exist in the location `E:\\cprogram`. The first function creates a new file named `newprogram.txt` and opens it for writing as per the mode `'w'`.
The writing mode allows you to create and edit (overwrite) the contents of the file.
- Now let's suppose the second binary file `oldprogram.bin` exists in the location `E:\\cprogram`. The second function opens the existing file for reading in binary mode `'rb'`.
The reading mode only allows you to read the file, you cannot write into the file.

Opening Modes in Standard I/O		
Mode	Meaning of Mode	During Inexistence of file
<code>r</code>	Open for reading.	If the file does not exist, <code>fopen()</code> returns NULL.
<code>rb</code>	Open for reading in binary mode.	If the file does not exist, <code>fopen()</code> returns NULL.
<code>w</code>	Open for writing.	If the file exists, its contents are overwritten. If the file does not exist, it will be created.
<code>wb</code>	Open for writing in binary mode.	If the file exists, its contents are overwritten. If the file does not exist, it will be created.
<code>a</code>	Open for append. Data is added to the end of the file.	If the file does not exist, it will be created.
<code>ab</code>	Open for append in binary mode. Data is added to the end of the file.	If the file does not exist, it will be created.
<code>r+</code>	Open for both reading and writing.	If the file does not exist, <code>fopen()</code> returns NULL.
<code>rb+</code>	Open for both reading and writing in binary mode.	If the file does not exist, <code>fopen()</code> returns NULL.

<code>w+</code>	Open for both reading and writing.	If the file exists, its contents are overwritten. If the file does not exist, it will be created.
<code>wb+</code>	Open for both reading and writing in binary mode.	If the file exists, its contents are overwritten. If the file does not exist, it will be created.
<code>a+</code>	Open for both reading and appending.	If the file does not exist, it will be created.
<code>ab+</code>	Open for both reading and appending in binary mode.	If the file does not exist, it will be created.

Closing a File

The file (both text and binary) should be closed after reading/writing.
Closing a file is performed using the `fclose()` function.

```
fclose(fptr);
```

Here, `fptr` is a file pointer associated with the file to be closed.

Reading and writing to a text file

For reading and writing to a text file, we use the functions `fprintf()` and `fscanf()`. They are just the file versions of `printf()` and `scanf()`. The only difference is that `fprintf()` and `fscanf()` expects a pointer to the structure `FILE`.

Example 1: Write to a text file

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int num;
    FILE *fptr;

    // use appropriate location if you are using MacOS or Linux
    fptr = fopen("C:\\\\program.txt", "w");

    if(fptr == NULL)
    {
        printf("Error!");
        exit(1);
    }

    printf("Enter num: ");
    scanf("%d", &num);

    fprintf(fptr, "%d", num);
    fclose(fptr);

    return 0;
}
```

This program takes a number from the user and stores in the file program.txt. After you compile and run this program, you can see a text file program.txt created in C drive of your computer. When you open the file, you can see the integer you entered.

Example 2: Read from a text file

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int num;
    FILE *fptr;

    if ((fptr = fopen("C:\\program.txt", "r")) == NULL){
        printf("Error! Opening file");

        // Program exits if the file pointer returns NULL.
        Exit(1);
    }

    fscanf(fptr, "%d", &num);

    printf("Value of n=%d", num);
    fclose(fptr);

    return 0;
}
```

This program reads the integer present in the program.txt file and prints it onto the screen.

If you successfully created the file from Example 1, running this program will get you the integer you entered.

Other functions like `fgetchar()`, `fputc()` etc. can be used in a similar way.

Reading and writing to a binary file

Functions `fread()` and `fwrite()` are used for reading from and writing to a file on the disk respectively in case of binary files.

Writing to a binary file

To write into a binary file, you need to use the `fwrite()` function. The functions take four arguments:

1. address of data to be written in the disk
2. size of data to be written in the disk
3. number of such type of data
4. pointer to the file where you want to write.

```
fwrite(addressData, sizeData, numbersData, pointerToFile);
```

Example 3: Write to a binary file using `fwrite()`

```
#include <stdio.h>
#include <stdlib.h>
struct threeNum
{
    int n1, n2, n3;
};
int main()
{
    int n;
    struct threeNum num;
    FILE *fptr;
    if ((fptr = fopen("C:\\program.bin", "w")) == NULL){
        printf("Error! opening file")
    }
    // Program exits if the file pointer returns NULL.
    exit(1);
}
for(n = 1; n < 5; ++n)
{
    num.n1 = n;
    num.n2 = 5*n;
    num.n3 = 5*n + 1;
    fwrite(&num, sizeof(struct threeNum), 1, fptr);
}
fclose(fptr);
return 0;
}
```

Example 4: Read from a binary file using `fread()`

```
#include <stdio.h>
#include <stdlib.h>

struct threeNum
{
    int n1, n2, n3;
};

int main()
{
    int n;
    struct threeNum num;
    FILE *fptr;

    if ((fptr = fopen("C:\\program.bin", "rb")) == NULL){
        printf("Error! opening file");

        // Program exits if the file pointer returns NULL.
        exit(1);
    }

    for(n = 1; n < 5; ++n)
    {
        fread(&num, sizeof(struct threeNum), 1, fptr);
        printf("n1: %d\\tn2: %d\\tn3: %d\\n", num.n1, num.n2, num.n3);
    }
    fclose(fptr);

    return 0;
}
```

Problems